

Sistema de Gestión y Monitoreo de Invernaderos

OLIMPIADA NACIONAL DE ETP 2026

Olimpiada Nacional de Informática 2026 — Instancia Institucional

EEST N°2 — Berisso, Provincia de Buenos Aires

SISTEMA DE GESTIÓN Y MONITOREO DE INVERNADEROS

Automatización, informatización y control de temperatura y humedad del parque ambiental

CUE: 0611351-00

Escuela de Educación Secundaria Técnica N° 2

Berisso - Provincia de Buenos Aires

Profesor responsable

Daniel Serganczuk

Tramo Pedagógico en Secundaria Técnica - Técnica en Informática personal y profesional

Cargo: Profesor - Jefe de Departamento

Correo: deserganczuk@gmail.com Teléfono: +54 9 221 575-6796

Estudiantes integrantes del equipo

Villanueva Amparo — Informática — Curso 7ma 5ta — Satisfactorio

Villanueva Delfina — Informática — Curso 7ma 5ta — Satisfactorio

Bazan Thiago — Informática — Curso 7ma 5ta — Satisfactorio

Labaroni Francisco — Informática — Curso 7ma 5ta — Satisfactorio

Gomez Lautaro — Informática — Curso 7ma 5ta — Satisfactorio

Schreiber Sebastián — Informática — Curso 7ma 5ta — Satisfactorio

Archivo: INFO-TEC2-7-01.pdf

Índice

- 1. Introducción y problematización**
- 2. Análisis de la situación problemática**
- 3. Diseño de la solución y arquitectura**
- 4. Hardware: ESP32, sensor DHT11 y buzzer**
- 5. Base de datos (PostgreSQL)**
- 6. Backend (Node.js + Express)**
 - â€¢ 6.1. Estructura general del servidor
 - â€¢ 6.2. Autenticación y sesiones
 - â€¢ 6.3. Dashboard
 - â€¢ 6.4. Empleados y Usuarios
 - â€¢ 6.5. Llamados
 - â€¢ 6.6. Parámetros (rangos de alerta)
 - â€¢ 6.7. Reportes y exportación PDF/CSV
 - â€¢ 6.8. API para el ESP32
 - â€¢ 6.9. notificaciones.js (Email + WhatsApp)
- 7. Frontend (vistas EJS)**
- 8. Procesos de trabajo y fases del proyecto**
- 9. Despliegue en Render y credenciales**
- 10. Cumplimiento de los requerimientos de la consigna**
- 11. Informe final: criterios técnicos, ventajas y desventajas**
- 12. Registro de experiencia del equipo**
- 13. Bibliografía (APA 7)**
- 14. Anexo: capturas, croquis y material complementario**

1. Introducción y problematización

1.1. Contexto

El Parque Ambiental cuenta con varios invernaderos donde se cultivan plantas por debajo del nivel del mar. Estas plantas dependen de dos variables ambientales: la

1.2. Necesidades detectadas

Durante el relevamiento inicial se detectaron los siguientes problemas:

- **Falta de información en tiempo real:** no existía ningún instrumento que midiera de forma continua la temperatura y la humedad dentro de cada invernadero. Las mediciones se hacían, en el mejor de los casos, con termómetros manuales y no quedaba registro.
- **No había alertas:** si de noche o durante un fin de semana la temperatura salía del rango adecuado, nadie se enteraba hasta que era tarde.
- **Llamados de los empleados sin registro:** cuando un empleado detectaba un problema (falta de riego, macetas, etc.) se resolvía de palabra, sin quedar constancia.
- **Sin estadísticas:** no existía forma de saber cuántos llamados de emergencia hubo por mes, por área o por tipo de sensor.

1.3. Problematización

¿Cómo podemos **medir de forma continua, registrar y alertar** en tiempo real sobre la temperatura y la humedad de los invernaderos, y al mismo tiempo ordenar el trabajo de los empleados y generar reportes que sirvan para la toma de decisiones del parque?

De esa pregunta central derivan los requerimientos que la consigna plantea:

- Registrar áreas, empleados y usuarios con roles.
- Generar **llamados** (normal o emergencia), atenderlos y llevar su historial.
- **Alertar** por Email y WhatsApp cuando una lectura sale de rango o un empleado reporta una emergencia.
- Mostrar **reportes** filtrados por área, tipo y fecha, con **gráficos** y exportación a **PDF y CSV**.
- Publicar el sistema en un **servidor web** accesible con usuario y contraseña.

2. Cada 30 segundos envía las lecturas al servidor con `POST /api/lecturas`.
3. El servidor **guarda** la lectura en la tabla `lecturas`.
4. El servidor **compara** la lectura contra el rango configurado en `rangos_config` (la pantalla **Parámetros**).
5. Si la lectura sale de rango:
 • genera un **llamado de emergencia** (origen "sensor"),
 • **notifica** por Email y WhatsApp,
 • el ESP32 **suenan el buzzer** (porque la respuesta del servidor trae `alerta: true`).
6. Un empleado también puede generar llamados manuales (origen "empleado").
7. Los llamados se atienden desde la pantalla correspondiente y quedan en el historial.
8. Los **reportes** procesan ese historial con filtros, gráficos y exportación a PDF/CSV.

3.3. Roles y permisos

Rol	Puede ver	No puede ver
administrador ('rol = 'administrador')	Todo: áreas, llamados, reportes, empleados, usuarios, parámetros	— (acceso irrestricto)
genérico / empleado ('rol = 'generico')	Áreas, llamados, reportes	Empleados, usuarios, parámetros

En el código esto se controla con dos `*middlewares*`: `requiereLogin` y `requiereAdmin` (ver sección 6.2).

4. Hardware: ESP32, sensor DHT11 y buzzer

4.1. Componentes

Componente	Función
Placa ESP32 (Espressif)	Microcontrolador con WiFi integrado. Es el "cerebro" que lee sensores y se comunica con el servidor.
Sensor DHT11	Mide temperatura y humedad relativa. Digital, un solo pin de datos.
Buzzer activo	Suena una alarma cuando el servidor responde que una lectura está fuera de rango.

4.2. Conexión eléctrica (croquis)

```

ESP32                                DHT11
% % % % % % % %                    % % % % % % % %
% 3V3 % % % % % % % % % % % % % % $ VCC %
% GND % % % % % % % % % % % % % % $ GND %
% D4  % % % % % % % % % % % % % % $ DATA % (sensor DHT11)
% % % % , % % % %                    % % % % % % % %
  
```


Función `sonarAlarma()` — hace sonar el buzzer 5 veces (2.200 Hz, 200 ms cada tono).

Función `beepInicio()` — 3 pitidos cortos al encender la placa, para verificar que el buzzer funciona.

Función `setup()` — configura el monitor serie, el pin del buzzer, el sensor DHT11 y llama a `conectarWiFi()`.

Función `loop()` — se ejecuta para siempre:

```
if (millis() - ultimaLectura >= INTERVALO_MS) { // cada INTERVALO_MS (30 seg)
    float temp = dht.readTemperature(); // lee temperatura
    float hum = dht.readHumidity(); // lee humedad
    ...
    enviarLectura(SENSOR_ID_TEMP, temp, &alertaTemp); // envía temperatura
    enviarLectura(SENSOR_ID_HUM, hum, &alertaHum); // envía humedad
    if (alertaTemp || alertaHum) sonarAlarma(); // si algo salió de rango, suena
}
```

`millis()` es el tiempo que lleva la placa encendida. Si el tiempo de espera es menor a 30 segundos, solo con el último envío se logra un "temporizador" sin bloquear el programa.

5. Base de datos (PostgreSQL)

5.1. Modelo de datos

La base llamada `invernaderos` tiene 7 tablas. Este es el esquema y sus relaciones:

Tabla	Campos principales	Relación
`areas`	`id`, `nombre`, `tipo`, `descripcion`	Padres de empleados, sensores y llamados.
`empleados`	`id`, `nombre`, `apellido`, `dni`, `telefono`, `email`, `tarea`, `area_id`	`area_id` □ `areas.id`. Un empleado trabaja en un área.
`usuarios`	`id`, `empleado_id`, `username`, `password_hash`, `rol`, `activo`	`empleado_id` □ `empleados.id`. Cada usuario está vinculado a un empleado.
`sensores`	`id`, `area_id`, `tipo`, `codigo_dispositivo`, `activo`	`area_id` □ `areas.id`. Un área puede tener varios sensores.
`rangos_config`	`id`, `area_id`, `tipo_sensor`, `valor_min`, `valor_max`	Define el rango "normal" por área y tipo (temperatura/humedad).
`lecturas`	`id`, `sensor_id`, `valor`, `fecha_hora`	`sensor_id` □ `sensores.id`. Es la tabla que más crece.
`llamados`	`id`, `area_id`, `origen`, `sensor_id`, `empleado_id`, `tipo`, `descripcion`, `atendido`, `atendido_por`, `fecha_hora`, `fecha_atencion`	Puede originarse en un sensor o en un empleado.

5.2. Decisiones de diseño

⚠️ **Seguridad de contraseñas:** nunca se guardan en texto plano. Se guarda un hash Bcrypt

(10 rondas). En el login se compara el hash, no la contraseña.

• **Integridad referencial:** las claves foráneas (^{REFERENCES}) evitan llamados a sensores inexistentes o empleados borrados por accidente.

• **Índices:** se crearon índices sobre las columnas que más se filtran, lecturas(sensor_id, fecha_hora), llamados(area_id, fecha_hora) y llamados(tipo), para que las consultas de reportes sigan siendo rápidas cuando haya muchos datos.

5.3. Cómo se crea el esquema automáticamente

El archivo `esquema_invernaderos.sql` contiene los `db - CREATE TABLE`, que: . El problema era que podía haber errores al crearse en un servidor limpio, así que se escribió

1. Lee el archivo SQL, lo separa por punto y comay quita las líneas de comentario

(`--` ...). Este fue un problema antes de las sentencias que venían precedidas de un comentario se filtraban por error y las tablas

2. Ejecuta cada sentencia; si la tabla ya existe (error ^{42P07}) lo informa sin romper

(el script es **idempotente**: se puede correr mil veces).

3. Si la tabla de usuarios está vacía, crea el administrador inicial usando las variables

`ADMIN_USER` y `ADMIN_PASSWORD`. Si ya existe, **sincroniza** su contraseña con `ADMIN_PASSWORD` (así, cambiar la variable y redeployar cambia la clave).

4. Si no hay áreas cargadas, ejecuta `seed.sql` (datos de ejemplo: áreas, empleados, sensores, rangos, llamados y lecturas) para que el sistema muestre contenido apenas arranca.

6. Backend (Node.js + Express)

6.1. Estructura general del servidor (`server.js`)

El servidor arranca así:

```
require('dotenv').config();           // carga las variables de .env
const express = require('express');    // framework web
const session = require('express-session'); // sesiones por cookie
const bcrypt = require('bcrypt');      // hash de contraseñas
const pool = require('./db');          // conexión a PostgreSQL
const { notificarAlerta } = require('./notificaciones'); // alertas Email/WhatsApp
```


Se configuran las vistas **EJS** y los parsers de datos entrantes:

```
app.set('view engine', 'ejs'); // plantillas .ejs
app.use(express.static('public')); // CSS y archivos estáticos
app.use(express.urlencoded({ extended: true })); // formularios del navegador
app.use(express.json()); // JSON que manda el ESP32
```

La sesión guarda al usuario logueado (`req.session.usuario`):

```
app.use(session({
  secret: process.env.SESSION_SECRET || 'secreto_temporal',
  resave: false,
  saveUninitialized: false,
}));
```

6.2. Middlewares de acceso

```
function requiereLogin(req, res, next) {
  if (!req.session.usuario) return res.redirect('/'); // no logueado !' login
  next(); // si está logueado, continúa con la ruta
}
```

```
function requiereAdmin(req, res, next) {
  if (!req.session.usuario) return res.redirect('/');
  if (req.session.usuario.rol !== 'administrador') {
    return res.status(403).send('No tenés permiso...'); // empleado !' 403
  }
  next();
}
```

Cada ruta usa el middleware según su privacidad: `requiereLogin` para cualquier usuario y `requiereAdmin` para las de administración (empleados, usuarios, parámetros).

6.3. Login ('POST /login')

1. Busca al usuario por su `username` y que esté `activo = true`.
2. Compara la contraseña ingresada contra el hash con `bcrypt.compare`.
3. Si coincide, guarda el usuario en la sesión y redirige a `/dashboard`.

```
const passwordOk = await bcrypt.compare(password, usuario.password_hash);
if (!passwordOk) {
  return res.render('login', { error: 'Usuario o contraseña incorrectos' });
}
```

```

}
req.session.usuario = usuario;
res.redirect('/dashboard');

```

6.4. Dashboard (`GET /dashboard`)

Es la pantalla principal de consulta de datos para traer, de cada área, la última lectura de temperatura y la

```

LEFT JOIN LATERAL (
  SELECT l.valor FROM lecturas l
  JOIN sensores s ON s.id = l.sensor_id AND s.area_id = a.id AND s.tipo = 'temperatura'
  ORDER BY l.fecha_hora DESC LIMIT 1
) temp ON true

```

LIMIT 1 con ORDER BY fecha_hora DESC = "la más reciente". Además trae hasta 10 llamados recientes para mostrarlos debajo de las tarjetas de áreas.

6.5. Empleados y Usuarios

â€¢ `GET/POST /empleados` — listar y crear fichas de empleados (nombre, apellido, DNI, teléfono, email, tarea y área). Los datos de la ficha se cargan desde el panel.

â€¢ `POST /empleados/:id/editar` y `/empleados/:id/eliminar` — edición y baja, con UPDATE DELETE y sobre la tabla empleados.

â€¢ `GET/POST /usuarios` — crear usuarios con username, contraseña (hasheada con bcrypt.hash(password, 10)) y rol.

â€¢ `POST /usuarios/editar/:id` — edita el usuario y, si tiene empleado_id, actualiza también los datos del empleado vinculado, o crea el empleado nuevo si no tenía uno. Esto cumple los puntos "configuración del sistema" y "fichas de empleados" de la consigna.

6.6. Llamados

â€¢ `GET /llamados` — lista todos los llamados con filtros por tipo y área. La consulta arma el WHERE dinámicamente con parámetros \$1, \$2... (protección contra inyección SQL).

â€¢ `POST /llamados/:id/atender` — marca el llamado como atendido, registra quién lo atendió (atendido_por) y la fecha (fecha_atencion = NOW()).

â€¢ `POST /llamados` — un empleado genera un llamado manual. Si el tipo es emergencia,

también dispara las notificaciones (`notificarAlerta`).

```
if (tipo === 'emergencia') {
  notificarAlerta({ area, tipo: 'emergencia', origen: 'empleado', descripcion })
}
```

6.7. Parámetros (pantalla que configura las alertas)

La tabla `rangos_config` guarda, por área y por tipo de sensor, el **mínimo** y el **máximo** que se consideran normales.

â€¢ **GET/POST /parametros** — lista y agrega parámetros.

â€¢ **POST /parametros/:id/editar** y **/eliminar** — cambian o quitan un rango.

Estos rangos son los que administra el ESP32 configurado en el sistema (para el empleado y el propietario). Por eso la pantalla es una alarma cambiando los límites).

6.8. Reportes y exportación ('GET /reportes' y '/reportes/exportar')

â€¢ **Filtros:** área, tipo (normal/emergencia) y rango de fechas desde/hasta. La función

`buildFiltrosLLamados()` arma el `WHERE` con los filtros presentes.

â€¢ **Resumen:** `GROUP BY tipo, atendido` □ cuenta cuántos llamados hay por tipo y estado.

â€¢ **Gráficos:** la vista `reportes.ejs` incorpora **Chart.js** desde un CDN y muestra un

gráfico de **líneas** (evolución de temperatura y humedad) y un gráfico de **torta/dona** (distribución de estados o riego).

â€¢ **Exportación CSV:** arma el CSV manualmente con `'` como separador, encabezados en español y BOM UTF-8 (`\uFEFF`) para que Excel muestre bien los acentos.

â€¢ **Exportación PDF:** usa la librería **PDFKit** (`require('pdfkit')`). Dibuja una tabla con encabezados verde oscuro, repetidos en cada página si la tabla se corta, y un total final.

```
if (formato === 'csv') { ... return res.send('\uFEFF' + csv); }
// PDF
const doc = new PDFDocument({ size: 'A4', margin: 50 });
doc.pipe(res);
```

6.9. API para el ESP32 ('POST /api/lecturas')

Es una ruta **pública** (sin login) porque la consume un dispositivo:

```
app.post('/api/lecturas', async (req, res) => {
  const { sensor_id, valor } = req.body;
  if (!sensor_id || valor === undefined)
    return res.status(400).json({ error: 'Faltan sensor_id o valor' });
```

1. Guarda la lectura: `INSERT INTO lecturas (sensor_id, valor)`.

2. Consulta de qué área y tipo es ese sensor.

3. Consulta el rango configurado para esa área/tipo en `rangos_config`.

4. Si el valor está fuera del rango:

â€¢ alerta = true

â€¢ inserta un llamado de emergencia (origen 'sensor'),

â€¢ dispara `notificarAlerta(...)`.

5. Responde `{ ok: true, alerta }` — con la alerta en `true`, el ESP32 hace sonar el buzzer.

```
res.status(201).json({ ok: true, alerta });
```

6.10. 'notificaciones.js' — alertas por Email y WhatsApp

La función `notificarAlerta(datos)` arma un mensaje de texto único y lo envía en paralelo:

```
const [email, whatsapp] = await Promise.all([enviarEmail(...), enviarWhatsApp(...)]);
```

â€¢ **Email (SMTP):** usa `nodemailer`. Si la variable `SMTP_HOST` no está configurada, responde 'no configurado' y no falla (todo es opcional).

â€¢ **WhatsApp (Twilio):** llama a la API de Twilio con `fetch`, autenticándose con `Authorization: Basic` (SID y token codificados en base64). El sandbox gratis de Twilio permite enviar a tu propio número.

Ambos se envían con `Promise.all`, en paralelo, para que el llamado no se demore.

7. Frontend (vistas EJS)

Las vistas están en la carpeta `views/` y usan la plantilla **EJS** (HTML con código JS embebido con `<% %>`).

Vista	Función
'login.ejs'	Pantalla de ingreso con usuario y contraseña.

`dashboard.ejs`	Tarjetas por área con la última temperatura/humedad + últimos 10 llamados.
`llamados.ejs`	Listado con filtros y botón para marcar como atendido.
`reportes.ejs`	Resumen estadístico, filtros, gráficos (Chart.js) y exportación.
`empleados.ejs`	Alta, edición y baja de fichas de empleados.
`usuarios.ejs`	Gestión de usuarios y sus vínculos con empleados.
`parametros.ejs`	Configurar los rangos de alerta por área y tipo de sensor.

Ejemplo de EJS en el menú lateral (aparece lo que corresponde según el rol):

```
<% if (usuario.rol === 'administrador') { %>
  <a href="/empleados">Empleados</a>
  <a href="/usuarios">Usuarios</a>
  <a href="/parametros">Parámetros</a>
<% } %>
```

El CSS (`public/css/style.css`) usa variables de color para una paleta verde "parque", con tipografía Work Sans + Fraunces de Google Fonts.

8. Procesos de trabajo y fases del proyecto

El equipo trabajó simulando una consultora de software. Las fases fueron:

Fase 1 — Relevamiento y problematización Entrevista de necesidades con docentes y análisis de la consigna. Se definió qué medir (temperatura y humedad) y qué gestionar (empleados, llamados, reportes).

Fase 2 — Diseño de la solución Se decidió la arquitectura (ESP32 + API + base + web), el modelo de datos y los roles.

Fase 3 — Desarrollo

• Backend: servidor Express con rutas de gestión, reportes y API.

• Base: esquema SQL, seed de datos y script idempotente `db-setup.js`.

• Frontend: vistas EJS y CSS.

• Hardware: programa del ESP32 (DHT11, WiFi, buzzer).

• Alertas: Email y WhatsApp.

Fase 4 — Pruebas Pruebas locales con Postman (API), login con ambos roles, filtros de llamados, exportación a PDF/CSV y tablas de resumen (Se detectaron y corrigieron bugs (por ejemplo el de

Fase 5 — DesplieguePublicación del servidor en Render con base de datos PostgreSQL gestionada y despliegue automático desde GitHub.

Organización del registro de trabajo:el trabajo se versionó enGity se subió aGitHub(repositorio privado), lo que permitió suma colaborativa y recuperación ante errores. Cada integrante tuvo roles definidos (ver Registro de experiencia).

9. Despliegue en Render y credenciales

9.1. Plataforma de despliegue

Se eligióRenderporque ofrece un plan gratuito con:

• **Web service**Node.js con deploy automático desde GitHub.

• **PostgreSQL gestionado**conectado por DATABASE_URL.

El archivo render.yaml (Blueprint) describe toda la infraestructura: el servicio web invernaderos-app y la base invernaderos-db. Al ejecutarlo se crean:

```
node db-setup.js && npm start
```

db-setup.js crea el esquema, el administrador (con ADMIN_PASSWORD) servidor web público de acceso por usuario y clave (Requisitos de la consigna: el sistema debe estar en un

9.2. Credenciales de acceso (obligatorio incluir en la presentación)

Dato	Valor
URL pública	'https://invernaderos-app.onrender.com'
Usuario administrador	'admin'
Contraseña	*(definida en 'ADMIN_PASSWORD'; completar con la que se fijó en Render)*
Usuario genérico (de ejemplo)	'jperez'
Contraseña (si se cargó el seed)	'admin123'

Nota:se debe completar la contraseña del admin antes de la presentación.

10. Cumplimiento de los requerimientos de la consigna

Requerimiento de la consigna	Dónde se cumple
A.1 — Configuración de áreas/zonas y asignación de admin/empleados	Tabla 'areas'; alta de empleados/usuarios en '/empleados' y '/usuarios'; 'rangos_config' en '/parametros'.
A.1 — Ficha de empleados	Alta/edición con DNI, teléfono, email, tarea y área.
A.1 — Creación y edición de usuarios (Admin irrestricto / Genérico parcial)	Tabla 'usuarios' con 'rol'; validado por 'requiereAdmin'.
A.1 — Cantidad y tipo de llamados, atendidos y no atendidos	Tabla 'llamados' + pantalla que los lista y permite atenderlos.

A.1 — Visualizador de reportes en tablas o gráficos (barras, pastel, líneas)	Resumen en tabla + gráficos de líneas y torta (Chart.js).
A.1 — Filtrado por área, origen, fecha y hora	Filtros de `/reportes` y `/llamados`.
A.1 — Exportación PDF o CSV	Botones en `/reportes/exportar`.
B — Aplicación web con usuario y contraseña	Login por sesión, vista responsive desde el celular.
B — Ver alertas desde el celular	Avisos por Email y WhatsApp + alertas visibles en la web.
C — Servidor público con usuario/clave	Desplegado en Render, acceso por usuario y contraseña.
D — Informe final con criterios técnicos, ventajas/desventajas y bibliografía	Este documento (secciones 11 y 13).

11. Informe final: criterios técnicos, ventajas y desventajas

11.1. Criterios técnicos de selección

El equipo eligió una herramienta que cumpla con los siguientes criterios: **comunidad grande (casapopen software libre) y gratuito, fácil de desplegar, documentación abundante**

Herramienta	Por qué se eligió
Node.js + Express	JavaScript en todo el stack (mismo lenguaje en servidor y vista), muy rápido de desarrollar, gran ecosistema.
PostgreSQL	Base relacional robusta, gratis, con integridad referencial y buen soporte de fechas para reportes.
EJS	Plantillas simples: reutiliza el propio HTML aprendido en la escuela.
ESP32 + DHT11	Placa con WiFi integrado (no hace falta un módulo aparte) y sensor digital barato.
Chart.js	Gráficos de líneas/torta con pocas líneas de código, vía navegador.
PDFKit	Generación de PDF en el propio servidor sin programas externos.
Nodemailer	Envío de correos SMTP (compatible con Gmail).
Twilio	WhatsApp desde una API simple, con plan gratis de desarrollo.
Render	Plan gratuito, deploy automático desde GitHub, PostgreSQL gestionado.

11.2. Ventajas y desventajas operativas

Ventajas

â€¢ **Monitoreo en tiempo real** (lectura cada 30 segundos) sin depender de una persona.

â€¢ **Alertas automáticas** por Email/WhatsApp cuando algo sale de rango.

â€¢ **Historial completo**: llamados con fecha, tipo y quién atendió.

â€¢ **Reportes con exportación** listos para presentar.

â€¢ **Roles** protegen la configuración crítica.

â€¢ **Despliegue automático**: cada cambio en GitHub actualiza el sistema.

Desventajas / limitaciones

• Depende de la conexión Wi-Fi del lugar: si se corta, el ESP32 no envía lecturas (el código lo reintenta solo).

• El sandbox de WhatsApp de Twilio solo puede escribir a números verificados (para producción hay que pagar).

• El plan gratuito de Render duerme el servicio si no se usa un tiempo (se despierta solo al entrar, normalmente en segundos).

• El sensor DHT11 no es de alta precisión; para uso "profesional" serviría un DHT22 o un BME280 (con el mismo código se puede cambiar).

• Los gráficos actuales usan datos de ejemplo; podrían conectarse a las lecturas reales de la base para una versión futura.

12. Registro de experiencia del equipo

(Máximo 1 carilla. Este texto es una plantilla: cada integrante debe completarlo con las vivencias reales del grupo. Recomendado: 3-4 párrafos.)

- El equipo se organizó repartiendo responsabilidades: mientras unos se enfocaron en la
- programación del servidor y la base de datos, otros trabajaron la placa ESP32, el armado
- del sensor y el diseño de las vistas. Trabajamos en conjunto para... (completar).

Preguntas para guiar la redacción:

- ¿Cómo se organizaron los tiempos, la división de tareas y los roles?
- ¿Cómo funcionaron como equipo?
- ¿Cuáles fueron las principales dificultades para la resolución de la tarea?
- ¿Las pudieron resolver? ¿Cómo?

13. Bibliografía (APA 7)

Las referencias se citan según **Normas APA 7.ª edición** (autor/entidad, año, título, medio, URL y fecha de consulta).

1. Espressif Systems. (2023). *ESP32 Series Datasheet*. Recuperado el 9 de septiembre de 2026, de https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
2. Aosong Electronics. (s. f.). *DHT11 Humidity & Temperature Sensor Datasheet*. Recuperado el 9 de septiembre de 2026, de <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf>
3. Node.js. (s. f.). *Documentación de Node.js*. Recuperado el 9 de septiembre de 2026, de <https://nodejs.org/es/docs/>
4. Express – Node.js web framework. (s. f.). *Documentación oficial de Express*. Recuperado el 9 de septiembre de 2026, de <https://expressjs.com/es/>
5. PostgreSQL Global Development Group. (s. f.). *PostgreSQL Documentation*. Recuperado el 9 de septiembre de 2026, de <https://www.postgresql.org/docs/>
6. EJS. (s. f.). *Embedded JavaScript templating*. Recuperado el 9 de septiembre de 2026, de <https://ejs.co/>
7. Chart.js. (s. f.). *Chart.js documentation*. Recuperado el 9 de septiembre de 2026, de <https://www.chartjs.org/>
8. Keene, D. (2010-2024). *PDFKit — A JavaScript PDF generation library for Node.js*. Recuperado el 9 de septiembre de 2026, de <https://pdfkit.org/>
9. Nodemailer. (s. f.). *Nodemailer — Send e-mails with NodeJS*. Recuperado el 9 de septiembre de 2026, de <https://nodemailer.com/>
10. Twilio Inc. (s. f.). *WhatsApp Business Platform*. Recuperado el 9 de septiembre de 2026, de <https://www.twilio.com/whatsapp>
11. Render. (s. f.). *Render Documentation — Blueprints*. Recuperado el 9 de septiembre de 2026, de <https://render.com/docs/blueprint-spec>
12. bcrypt (npm). (s. f.). *bcrypt package*. Recuperado el 9 de septiembre de 2026, de <https://www.npmjs.com/package/bcrypt>
13. Arduino. (s. f.). *Arduino IDE y documentación*. Recuperado el 9 de septiembre de 2026, de <https://www.arduino.cc/>
14. Google Fonts. (s. f.). *Oswald y Work Sans*. Recuperado el 9 de septiembre de 2026, de <https://fonts.google.com/>

14. Anexo: capturas, croquis y material complementario

(En esta sección se insertarán las imágenes: capturas del dashboard, de la pantalla de llamados, de los reportes con gráficos, del mensaje de WhatsApp/Email recibido, del circuito del ESP32 y del video del proyecto. En el documento PDF cada imagen lleva su etiqueta "Figura N".)

â€¢ Figura 1: Circuito físico del ESP32 con DHT11 y buzzer.

â€¢ Figura 2: Dashboard con tarjetas de áreas.

â€¢ Figura 3: Pantalla de llamados.

â€¢ Figura 4: Reportes con gráficos (líneas y torta).

â€¢ Figura 5: Exportación PDF del reporte.

â€¢ Figura 6: Alerta recibida por WhatsApp/Email.

â€¢ Video del proyecto: subido a Google Drive con permiso de lectura (pegar aquí la URL).